Machine Learning: Questions and Answers

Experiment 1: PCA on the Wine Dataset

Q1. What is PCA and why is it used here?

PCA (Principal Component Analysis) is a dimensionality reduction technique. It converts the original features into a smaller set of new features called principal components. Here it reduces 13 wine measurements to 2 components so the data can be plotted and the wine groups inspected.

Q2. Why do we standardize the data before applying PCA?

PCA is based on variance, so a feature measured in large units would dominate the components. StandardScaler gives every feature mean 0 and standard deviation 1. This puts all features on the same scale so each one contributes fairly.

Q3. What does the explained variance ratio tell us?

It tells what fraction of the total variance each principal component captures. In this experiment the first two components together keep about 55% of the variance. A higher value means less information is lost in the reduction.

Q4. What is a principal component?

A principal component is a linear combination of the original features. The first component points in the direction of maximum variance in the data. Each next component is perpendicular to the previous ones and captures the most remaining variance.

Q5. Is PCA a supervised or unsupervised technique?

PCA is unsupervised because it does not use the class labels while computing the components. In this experiment the segment labels are used only afterwards, to colour the scatter plot and check the separation.

Q6. What did the scatter plot of PC1 vs PC2 show?

The three wine segments formed clearly separated groups in the 2D plot. This means the two components retained enough information to distinguish the groups. It confirms that PCA preserved the useful structure of the data.

Q7. What is dimensionality reduction and why is it needed?

Dimensionality reduction means representing the data with fewer features while keeping most of the useful information. It is needed because many features make models slower, harder to plot and prone to noise and overfitting. In the syllabus this comes under feature transformation in Unit I.

Q8. How is PCA different from LDA?

Both reduce dimensions, but PCA is unsupervised and finds directions of maximum variance without looking at labels. LDA is supervised and finds directions that separate the classes best. The syllabus lists PCA and LDA together as the two main dimensionality reduction techniques.

Q9. What is the role of eigenvectors and eigenvalues in PCA?

The principal components are the eigenvectors of the covariance matrix of the standardized data. The eigenvalue of each component tells how much variance that component carries. Components are sorted by eigenvalue, so the first component carries the most information.

Q10. How do we decide how many principal components to keep?

We look at the cumulative explained variance and keep enough components to cross a chosen level, commonly 90 to 95 percent. Another way is the scree plot, where we keep the components before the curve flattens. Here 2 components were kept mainly so the data could be plotted in 2D.

Q11. Does PCA select original features or create new ones?

PCA creates new features. Each principal component is a weighted combination of all the original features, not a subset of them. This is different from feature selection methods, which keep some original features and drop the rest.

Q12. What are the limitations of PCA?

PCA is a linear method, so it cannot capture curved or nonlinear structure in the data. The components are combinations of many features, so they are hard to interpret in real world terms. It is also sensitive to scaling, which is why standardization is done first.

Experiment 2: Uber Fare Prediction with Linear, Ridge and Lasso Regression

Q1. What is regression and what are we predicting here?

Regression is a supervised learning task that predicts a continuous value. Here the target is the Uber fare, predicted from distance, duration and number of passengers. The model learns the relationship between these features and the fare from past trips.

Q2. What is an outlier and how was it detected here?

An outlier is a value that is far away from the rest of the data. It was detected with the IQR rule: fares below Q1 minus 1.5 times IQR or above Q3 plus 1.5 times IQR are outliers. Such rows were removed because they can pull the regression line away from the general pattern.

Q3. What does correlation with fare tell us?

Correlation measures how strongly a feature moves together with the fare, from -1 to +1. Distance (about 0.99) and duration (about 0.95) are strongly related to fare, while passengers has almost no relation. Strong features usually make better predictors.

Q4. What is the difference between Ridge and Lasso regression?

Both add a penalty on the size of the coefficients to plain linear regression. Ridge uses the sum of squared coefficients (L2 penalty) and shrinks them towards zero. Lasso uses the sum of absolute

values (L1 penalty) and can shrink some coefficients exactly to zero, which also does feature selection.

Q5. Why is regularization needed at all?

Without regularization a model can overfit, meaning it fits the training data too closely and performs poorly on new data. The penalty keeps the coefficients small and the model simpler. This usually improves accuracy on unseen test data.

Q6. What do R2 and RMSE mean in the results?

R2 tells how much of the variation in fare the model explains; 1.0 is perfect and all three models scored about 0.986. RMSE is the average prediction error in fare units, about 14 rupees here. Lower RMSE and higher R2 mean a better model.

Q7. Is this simple linear regression or multiple linear regression?

It is multiple linear regression, because the fare is predicted from several features together: distance, duration and passengers. Simple linear regression uses only one input feature. The syllabus also separates univariate from multivariate and linear from nonlinear regression in the same way.

Q8. What are overfitting and underfitting?

Overfitting means the model learns the training data too closely, including its noise, and then fails on new data. Underfitting means the model is too simple and cannot capture the basic pattern, so it performs poorly even on training data. A good model sits between these two extremes.

Q9. What is the bias-variance tradeoff?

Bias is the error from a model being too simple, and variance is the error from a model being too sensitive to the training data. Simple models have high bias and low variance, complex models have low bias and high variance. The goal is a balance that gives the lowest total error on unseen data.

Q10. What is ElasticNet regression?

ElasticNet combines both penalties: the L1 penalty of Lasso and the L2 penalty of Ridge in one model. It is useful when many features are correlated, where Lasso alone may drop useful ones arbitrarily. The syllabus lists ElasticNet along with Ridge and Lasso as a regularized regression technique.

Q11. What are MSE and MAE, and how are they different from RMSE?

MSE is the average of squared errors and MAE is the average of absolute errors. RMSE is simply the square root of MSE, which brings the error back into the original units of the target. MAE treats all errors equally, while MSE and RMSE punish large errors more strongly.

Q12. What is adjusted R2 and when is it preferred over R2?

Plain R2 always increases when a new feature is added, even if the feature is useless. Adjusted R2 corrects for the number of features and only increases when a feature genuinely improves the

model. It is preferred when comparing multiple regression models with different numbers of features.

Experiment 3: SVM Classification of Handwritten Digits

Q1. What is a Support Vector Machine?

SVM is a supervised classification algorithm that finds the best boundary, called a hyperplane, between classes. It chooses the hyperplane with the maximum margin, that is the largest distance to the nearest points of each class. Those nearest points are the support vectors.

Q2. What data does the digits dataset contain?

It has 1797 images of handwritten digits, each image is 8 by 8 pixels. Flattened, each image becomes 64 numeric features. The label is the digit from 0 to 9, so it is a 10 class classification problem.

Q3. Why do we split the data into train and test sets?

The model is trained on the training set and evaluated on a separate test set it has never seen. This gives an honest estimate of how the model will perform on new data. Here 80% was used for training and 20% for testing.

Q4. What is a confusion matrix?

It is a table that compares actual classes with predicted classes for the test set. Correct predictions lie on the diagonal. Off diagonal entries show exactly which digits were confused with each other, for example a 8 predicted as a 1.

Q5. What do precision, recall and F1 score mean?

Precision is the fraction of predicted positives that are actually correct. Recall is the fraction of actual positives that the model found. F1 score is the harmonic mean of precision and recall and balances both in one number.

Q6. The accuracy is about 98.9%. What does that mean and is it good?

It means about 98.9% of the test images were classified into the correct digit. For a 10 class problem where random guessing gives 10%, this is a very strong result. It shows SVM works well on this kind of image data.

Q7. What is a kernel in SVM and which kernel is used here?

A kernel lets SVM draw curved boundaries by mapping the data into a higher dimensional space without computing that mapping directly. The syllabus lists the radial basis (Gaussian), polynomial and sigmoid kernels. This experiment uses the default RBF kernel of SVC, which handles the curved boundaries between digit classes well.

Q8. What is soft margin SVM and what does the parameter C do?

Hard margin SVM allows no point on the wrong side of the boundary, which fails when classes overlap. Soft margin SVM allows some violations, and C controls the tradeoff: a large C punishes

violations heavily and fits training data tightly, a small C keeps a wider margin and tolerates more errors. Soft margin is the practical version used in real data.

Q9. SVM is naturally a binary classifier. How does it handle 10 digit classes?

Two standard strategies are used for multiclass problems. One-vs-One trains a classifier for every pair of classes (45 classifiers for 10 classes) and takes a vote, while One-vs-All trains one classifier per class against the rest. The SVC implementation uses One-vs-One internally.

Q10. What is the difference between a balanced and an imbalanced classification problem?

In a balanced problem every class has roughly the same number of examples, like the digits data here, where each digit appears about equally. In an imbalanced problem one class dominates, and a model can show high accuracy just by predicting the majority class. That is why precision, recall and F1 per class matter more than plain accuracy there.

Q11. Why are the pixel values scaled before training the SVM?

SVM uses distances and margins, so a feature with larger values would dominate the geometry of the boundary. Scaling brings all 64 pixel features to a comparable range, which makes the margin fair and usually improves accuracy. Distance based and margin based methods in general need this step.

Q12. How is SVM different from K-Nearest Neighbours?

KNN is a lazy, non-parametric method: it stores all training data and classifies a new point by the majority label of its nearest neighbours. SVM is an eager, parametric style method: it learns one decision boundary during training and afterwards only the support vectors matter. SVM usually generalizes better on high dimensional data like these 64 pixel features.

Experiment 4: K-Means Clustering on the Iris Dataset

Q1. What is K-Means clustering?

K-Means is an unsupervised algorithm that groups data into k clusters. Each point is assigned to the nearest cluster centre (centroid), then the centroids are recomputed as the mean of their points. These two steps repeat until the clusters stop changing.

Q2. What is inertia in K-Means?

Inertia is the sum of squared distances of every point to its assigned cluster centre. Smaller inertia means tighter, more compact clusters. It is the quantity that K-Means tries to minimize.

Q3. What is the elbow method and what did it show here?

We run K-Means for k from 1 to 10 and plot inertia against k. Inertia falls fast at first and then slows down; the bend or elbow is the suggested number of clusters. Here the elbow is at k = 3, which matches the three iris species.

Q4. Why can we not just keep increasing k to reduce inertia?

Inertia always decreases as k grows, and with one cluster per point it becomes zero. But too many clusters lose the useful grouping and overfit the data. The elbow point balances compact clusters against a sensible number of groups.

Q5. K-Means is unsupervised, so how did we check the result?

The true species labels were hidden during clustering and used only for checking afterwards. One cluster contained all 50 setosa flowers, and the other two matched versicolor and virginica with only a few mixes. This purity check shows the clusters are meaningful.

Q6. Why is the choice of the initial centroids important?

K-Means starts from randomly placed centroids, and a poor start can end in a weak local solution. That is why the code uses n_init=10, running the algorithm 10 times from different starts. It keeps the run with the lowest inertia.

Q7. What is the silhouette method and how is it different from the elbow method?

The silhouette score of a point measures how close it is to its own cluster compared to the nearest other cluster, giving a value between -1 and +1. We pick the k with the highest average silhouette score. The elbow method only watches inertia falling, while silhouette also checks separation between clusters, so it is often clearer.

Q8. What are the main types of clustering in the syllabus?

Unit IV groups clustering into centroid-based methods like K-Means and K-Medoids, hierarchical methods (agglomerative and divisive), density-based methods like DBSCAN, and distribution-based methods like the Gaussian mixture model. Each family makes a different assumption about the shape of clusters. K-Means belongs to the centroid-based family.

Q9. How is K-Medoids different from K-Means?

K-Means represents each cluster by the mean of its points, which need not be an actual data point and is sensitive to outliers. K-Medoids represents each cluster by an actual data point, the medoid, which makes it more robust to outliers. The cost is that K-Medoids is slower on large data.

Q10. What are the advantages and disadvantages of K-Means?

Advantages: it is simple to understand, fast, and scales well to large datasets. Disadvantages: the number of clusters k must be fixed in advance, it is sensitive to initial centroids and to outliers, and it assumes clusters are roughly round and similar in size. These points are listed for K-Means in the syllabus itself.

Q11. When does the K-Means algorithm stop?

It stops when the cluster assignments no longer change between two iterations, which means the centroids have become stable. In practice a maximum iteration limit is also set (300 by default in sklearn) as a safety stop. At that point inertia has reached a local minimum.

Q12. What are some real applications of clustering?

The syllabus lists market segmentation, statistical data analysis, social network analysis, image segmentation and anomaly detection. A common example is grouping customers by income and spending so each group can be targeted differently. The iris grouping here is the same idea on flower measurements.

Experiment 5: Random Forest for Car Safety Prediction

Q1. What is a Random Forest?

Random Forest is an ensemble classifier made of many decision trees. Each tree is trained on a random subset of the data and a random subset of features. For prediction, all trees vote and the majority class is the final answer.

Q2. Why does a forest work better than a single decision tree?

A single tree can overfit and change a lot with small changes in data. Averaging the votes of many different trees cancels out individual errors. This reduces variance and usually gives higher and more stable accuracy.

Q3. The car dataset is all text categories. Why did we encode it?

Machine learning models work with numbers, not text labels like low, med or high. OrdinalEncoder converts each category into a number while keeping the natural order of values such as low < med < high < vhigh. The encoded matrix can then be given to the model.

Q4. What features and classes does the car dataset have?

It has 1728 cars described by 6 features: buying price, maintenance cost, doors, persons, luggage boot size and safety. The target has 4 classes: unacc, acc, good and vgood. The model predicts the acceptability class of a car from its features.

Q5. What does n_estimators=100 mean?

It is the number of decision trees in the forest, here 100 trees. More trees usually make the prediction more stable, up to a point. After enough trees the accuracy stops improving while training time keeps increasing.

Q6. The accuracy is about 98.8%. How do we read the classification report alongside it?

Accuracy is the overall fraction of correct predictions. The report gives precision, recall and F1 for every class separately, so a weak class cannot hide behind a good overall score. Here all four classes score high, so the model is reliable across classes.

Q7. What is ensemble learning and why is it needed?

Ensemble learning combines the predictions of several models into one final prediction. A single model may be weak or unstable, but a group of models makes different errors that partly cancel out. Unit V of the syllabus introduces ensembles exactly for this improvement in accuracy and stability.

Q8. What is bagging and how does Random Forest use it?

Bagging means training the same model on many bootstrap samples, that is random samples drawn with replacement from the data, and aggregating their outputs. Random Forest is bagging applied to decision trees, with one extra idea: each split also considers a random subset of features. Aggregation is by majority vote for classification.

Q9. How is boosting different from bagging?

In bagging the models are trained independently and in parallel on different samples. In boosting the models are trained one after another, and each new model focuses more on the examples the previous ones got wrong. AdaBoost, Gradient Boosting and XGBoost are the boosting methods named in the syllabus.

Q10. What is the difference between homogeneous and heterogeneous ensembles?

A homogeneous ensemble uses the same type of base model many times, like the 100 decision trees in this Random Forest. A heterogeneous ensemble combines different types of models, such as a tree, an SVM and a logistic regression together. Random Forest is a homogeneous ensemble.

Q11. Why does Random Forest reduce variance?

A single deep tree has low bias but high variance: small data changes can change the whole tree. Averaging many trees trained on different samples and feature subsets smooths out this instability, so variance falls while bias stays low. The syllabus lists variance reduction as a key strength of the Random Forest ensemble.

Q12. What is a voting ensemble?

A voting ensemble combines different models by letting them vote. In max voting the class predicted by most models wins, in averaging the predicted probabilities are averaged, and in weighted averaging stronger models get more weight. It is the simplest ensemble technique in the syllabus, before bagging and boosting.

Experiment 6: Maze Exploration with Q-Learning

Q1. What is reinforcement learning?

In reinforcement learning an agent learns by interacting with an environment through trial and error. It takes actions, receives rewards or penalties, and learns which actions lead to the highest total reward. Unlike supervised learning, no correct answers are given in advance.

Q2. What is Q-Learning and what is stored in the Q-table?

Q-Learning is a reinforcement learning algorithm that learns the value of every action in every state. These values are stored in a Q-table, here Q[row][col][action]. After training, the best action in a state is simply the one with the highest Q-value.

Q3. Explain the reward design used in the maze.

Each normal step gives -1, which pushes the agent to find a short path. Hitting a wall gives -5, which teaches the agent to avoid walls. Reaching the goal gives +20 and ends the episode, which is the main positive signal the agent works towards.

Q4. What is the epsilon-greedy policy?

With probability epsilon (0.15 here) the agent takes a random action to explore new possibilities. Otherwise it exploits, taking the action with the highest Q-value known so far. This balances learning new things against using what is already learned.

Q5. What roles do alpha and gamma play in the Q update?

Alpha (0.2) is the learning rate: how much a new experience changes the old Q-value. Gamma (0.95) is the discount factor: how much future rewards matter compared to immediate ones. A gamma close to 1 makes the agent value the final goal strongly.

Q6. How do we know the agent actually learned?

After 1500 training episodes the agent was run greedily, always picking the best known action. It reached the goal in 14 steps without any random moves. The printed maze shows the learned path from S to G avoiding all walls, which proves the Q-table captured a correct route.

Q7. What is a Markov Decision Process and the Markov property?

The Markov property says the future depends only on the present state, not on how the agent reached it. An MDP is the formal framework built on this: a set of states, actions, transition chances, rewards and a discount factor. The maze is a simple MDP because the next cell depends only on the current cell and the action chosen.

Q8. What is a policy and what is a value function?

A policy is the agent's strategy: it tells which action to take in each state. A value function estimates the total future reward expected from a state (or from a state-action pair) when following a policy. Q-Learning learns action values, and the final greedy policy simply picks the action with the highest value in each state.

Q9. What does the Bellman equation say?

It says the value of a state equals the immediate reward plus the discounted value of the next state, when the best action is taken. This splits one big problem into a step by step relation between neighbouring states. The Q-Learning update rule used in this experiment is built directly on this equation.

Q10. What are the basic elements of a reinforcement learning system?

The syllabus lists the agent, the environment, states, actions and rewards, along with the policy and value function. In this experiment the agent is the maze runner, the maze is the environment, each cell is a state, the four moves are actions, and the step, wall and goal scores are the rewards. Learning is the process of improving the policy from these rewards.

Q11. What type of reinforcement learning method is Q-Learning?

Q-Learning is a model-free, value-based method. It is model-free because the agent does not need to know the transition rules of the environment in advance; it learns purely from experience. It is value-based because it learns a value table first and derives the policy from it, instead of learning the policy directly.

Q12. What is the return in reinforcement learning, and what are real applications of RL?

The return is the total discounted reward collected from a state until the end of an episode; the agent's goal is to maximize it. The syllabus points to real time uses such as game playing agents (its case study builds a Tic-Tac-Toe player with Q-Learning), robotics and self-driving systems. Any task with sequential decisions and feedback suits RL.

Experiment 7: Emotion Detection from Text

Q1. What is the goal of this experiment?

The goal is to classify short text messages into one of three emotions: happy, sad or angry. It is a text classification task, so the text must first be converted into numbers that a classifier can use. Two classifiers are trained and compared.

Q2. What preprocessing is done on the text and why?

Each message is lowercased, URLs and mentions are removed, and punctuation is stripped. This reduces noise so that the same word in different forms, like Happy and happy, is treated as one feature. Cleaner text gives the model a smaller and more useful vocabulary.

Q3. What is TF-IDF?

TF-IDF converts text into numeric features. Term frequency counts how often a word appears in a message, and inverse document frequency lowers the weight of words that appear in almost every message. The result highlights words that are characteristic of a message, and using unigrams plus bigrams also captures word pairs.

Q4. What is the difference between Logistic Regression and Naive Bayes here?

Logistic Regression learns a weighted decision boundary between the classes and generally handles overlapping features well. Multinomial Naive Bayes uses word probabilities and assumes features are independent of each other given the class. That assumption is simple but often too strong for text, one reason it scored lower here.

Q5. What were the results and which model was better?

Logistic Regression reached about 0.87 accuracy and Naive Bayes about 0.80 on the test set. So Logistic Regression was the better model and was used for the confusion matrix and the demo. The confusion matrix shows most errors are a few messages mixed between emotion classes.

Q6. How was the better model tested on new sentences?

Three brand new sentences, never seen in training, were cleaned and converted with the same TF-IDF vectorizer. The model predicted happy, sad and angry correctly for the three demo

sentences. This shows the model generalizes to unseen text instead of only memorizing the training data.

Q7. Is this a binary or a multiclass classification problem?

It is a multiclass problem because there are three emotion classes: happy, sad and angry. The syllabus separates binary classification (two classes) from multiclass, and for multiclass the precision, recall and F1 are computed separately for each class. A macro average then summarizes them in one number per metric.

Q8. Naive Bayes belongs to which family of ML models?

Naive Bayes is a probabilistic model, one of the model families listed in Unit I along with geometric and logical models. It applies Bayes theorem to compute the probability of each class given the words in the message. The class with the highest probability is the prediction.

Q9. What is the difference between Bag of Words and TF-IDF?

Bag of Words simply counts how many times each word appears in a message, so common words like "the" and "is" get large counts. TF-IDF also uses counts but divides by how common the word is across all messages, so rare, meaningful words get more weight. That is why TF-IDF usually works better for classification.

Q10. Can a text classifier overfit? How would we know?

Yes. With many word features and limited training messages, the model can memorize the training text and fail on new wording. We know it is overfitting when training accuracy is much higher than test accuracy. The train test split in this experiment is exactly what exposes that gap.

Q11. Why check the per class report when overall accuracy looks fine?

If one emotion has far fewer messages than the others, the model can score good overall accuracy while doing badly on that rare class. The per class precision, recall and F1 in the classification report expose such a weak class. Unit III stresses per class evaluation for multiclass problems for this reason.

Q12. What happens if a new sentence contains words the model never saw in training?

Those words are simply ignored, because the TF-IDF vectorizer only has columns for the vocabulary learned from the training data. The prediction is made from the known words in the sentence. This is also why the same fitted vectorizer must be reused for new text, never a freshly fitted one.

General ML Viva Questions

Q1. What is machine learning and how is it different from traditional programming?

In traditional programming we write fixed rules and the program applies them to data to produce answers. In machine learning the program learns the rules itself from example data and then

applies them to new data. Unit I defines ML as systems that improve their performance on a task with experience, meaning with more data.

Q2. What are the main learning paradigms?

Supervised learning uses labelled data and covers regression and classification. Unsupervised learning uses unlabelled data and covers clustering and dimensionality reduction. Semi-supervised learning mixes a little labelled data with a lot of unlabelled data, and reinforcement learning learns from rewards and penalties through trial and error.

Q3. What is the difference between regression and classification?

Regression predicts a continuous number, like the Uber fare in Experiment 2. Classification predicts a category or label, like a digit, a car class or an emotion. Both are supervised tasks; the type of the target variable decides which one applies.

Q4. What are parametric and non-parametric models? Give examples.

Parametric models summarize the data in a fixed set of parameters, like linear regression and logistic regression, whose form does not grow with more data. Non-parametric models make fewer assumptions and grow with the data, like KNN and decision trees. Unit I lists these among the ways ML models are grouped.

Q5. What are geometric, probabilistic and logical models?

These are the model families described in Unit I. Geometric models use lines, planes and distances, for example SVM. Probabilistic models work with probabilities, for example Naive Bayes. Logical models use rules and tree structures, for example a decision tree, which is easy to read as if-else conditions.

Q6. What is overfitting and how can it be reduced?

Overfitting is when a model fits the training data so closely that it also learns the noise, and then performs poorly on new data. It can be reduced by using more training data, simplifying the model, applying regularization like Ridge or Lasso, and using ensembles like Random Forest. Checking performance on a separate test set is how overfitting is detected.

Q7. Explain the bias-variance tradeoff in simple words.

A model with high bias is too simple and misses the pattern (underfitting), while a model with high variance is too flexible and chases noise (overfitting). Total error is smallest at a middle level of complexity. This tradeoff, from Unit II, is the reason model selection is about balance, not maximum complexity.

Q8. Why do we split data into training and test sets?

Training data teaches the model, and test data, which the model never sees during training, gives an honest estimate of performance on new data. If we tested on training data itself, the score would look better than the model really is. A common split is 80 percent training and 20 percent testing, as used in these experiments.

Q9. When is accuracy a misleading metric?

Accuracy misleads when the classes are imbalanced. If 95 percent of examples belong to one class, a model that always predicts that class shows 95 percent accuracy while learning nothing useful. In such cases precision, recall and F1 score per class give the true picture, as stressed in Unit III.

Q10. Define precision, recall and F1 score.

Precision is the share of positive predictions that are actually correct. Recall is the share of actual positives that the model manages to find. F1 score is the harmonic mean of the two, useful when both false positives and false negatives matter. All three are read from the confusion matrix.

Q11. What is the difference between classification and clustering?

Classification is supervised: the model learns from labelled examples and assigns labels to new data. Clustering is unsupervised: no labels are given and the algorithm groups similar data points on its own, like K-Means grouping iris flowers in Experiment 4. The groups found by clustering then need interpretation by us.

Q12. How do bagging and boosting differ, in one line each?

Bagging trains many models independently on random samples of the data and averages or votes their answers to reduce variance, as Random Forest does. Boosting trains models one after another, each new model correcting the mistakes of the previous ones, as AdaBoost and Gradient Boosting do. Both are ensemble methods from Unit V.